

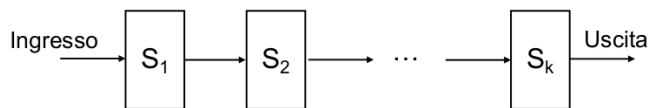
Cerchiamo di rendere l'esecuzione i nostri programmi : il multi ciclo visto finora non va male in quanto c'è anche il riuso dei componenti per svolgere più istruzioni diverse (spezzettamento istruzioni in micro istruzioni). Ma il problema di questa architettura è che è sequenziale in quanto esecuzione di un'istruzione si sviluppa secondo **fetch / decode / execute**. Quindi in generale con questo tipo di processore ci potrebbe un sottoutilizzo delle risorse, con singola esecuzione di istruzione nel ciclo. Andiamo a stimare questo tempo :

$$T_{exec} = n_{cicli} \cdot T_{clock} \text{ equivalente a } T_{exec} = \frac{n_{cicli}}{f_{clock}}$$

Dove ncicli (numero di cicli) si calcola così':

$$n_{cicli} = n_{insn} \cdot CPI$$

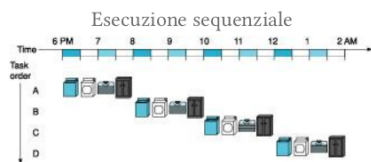
Dove CPI sta per clock process instruction (numero colpi clock necessari per eseguire istruzione). Vediamo ora come poter "aggiustare" / velocizzare queste equazioni : per esempio possiamo aumentare la frequenza di clock (legge di Moore) , ridurre il numero di istruzioni (differenza tra cisc e risc) . Andiamo ora a vedere l'architettura **PIPELINE** : architettura all'interno del quale vi è un "tubo", all'interno del quale scorrono successivamente varie istruzioni : **il processore esegue contemporaneamente varie parti delle istruzioni che sono nel tubo**. L'esecuzione di un'istruzione viene suddivisa in passi o stati : ogni istruzione li deve attraversare tutti. In dettaglio :



Vediamo un esempio :

Attività elementari per fare il bucato:

- Lavaggio 🧼
- Asciugatura 🧺
- Stiratura 🧺
- Riordino 🧺



Nella soluzione con pipeline viene avviato il ciclo di lavaggio successivo mentre quello precedente è ancora in esecuzione in un'altra fase



Quindi in dettaglio :

Class	Instruction	Syntax	Semantics
HW	Non-operational	nop	No operation (beyond the fetch phase)
L/A	Sum	addq %regsorg, %regdest	regdest = regsorg + regdest
	Subtraction	subq %regsorg, %regdest	regdest = regdest - regsorg
	Logic product	andq %regsorg, %regdest	regdest = regsorg and regdest
	Logic sum	orq %regsorg, %regdest	regdest = regsorg or regdest
	Logic negation	notq %register	register = not register
L/S	Loading of word	movq offset(%regbase), %regdest	regdest = memory[offset+regbase]
	Storage of word	movq %regsorg, offset(%regbase)	memory[offset+regbase] = regsorg
J	jump if flag X == 1	jX displacement	if flag X == 1 then RIP = RIP + displacement

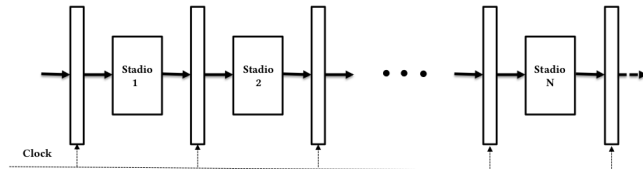
- Il formato istruzione è identico a quello del processore multiciclo (ma supportiamo solo istruzioni a 64 bit)

Per studiare una qualunque architettura pipeline , andiamo a vedere i passi elementari:

1. **Nop** :
 - a. Eseguo fetch, recupero dalla memoria , incremento rip , eseguo decode
 - b. Non devo fare niente per fare la NOP
2. **Istruzioni logico aritmetiche**
 - a. Prelevo il dato (fetch)
 - b. Eseguo fase decode : vedo gli operandi dell'istruzione logico-aritmetica
 - c. Possibili operandi sono solo i registri
 - d. Nella fase di exec : eseguo effettivamente l'operazione (ALU fa cose)
 - e. Dobbiamo scrivere il risultato nel registro (Write back)
3. **Load**: registro base + offset
 - a. Prelevo il dato (fetch)
 - b. Leggo registro base (trattato come registro sorgente) decode
 - c. Calcolo indirizzo da dove prelevare il dato (execute) -> offset+base
 - d. Leggo il dato della locazione di memoria puntato da offset+base -> memory
 - e. Scrivo il contenuto nel registro di destinazione -> write back
4. **Store**: scrittura in memoria ; non ho write back
 - a. Prelevo istruzione (fetch)

- b. Leggo registro base (trattato come registro sorgente) decode
 - c. Calcolo indirizzo da dove prelevare il dato (execute) -> offset+base
 - d. Leggo il dato della locazione di memoria puntato da offset+base -> memory
5. **Jmp** : salto condizionale
- a. Prelevo istruzione (fetch)
 - b. leggo registro base (trattato come registro sorgente) decode
 - c. Calcolo indirizzo da cui prelevare il dato , ricordando che il bit di flag è impostato a 1 -> execute
 - d. Aggiorno PC (program counter) in base al valore del flag

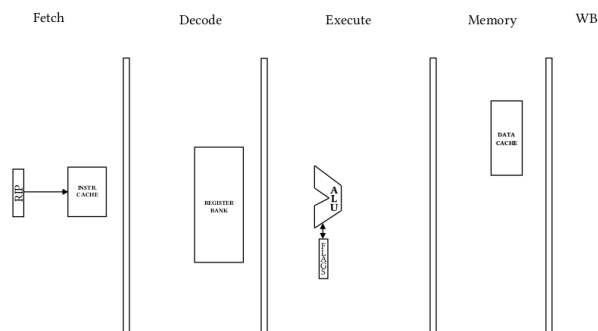
In generale si hanno da 2 a 5 fasi .Ma in questo caso per forza 5 fasi! Quindi il vantaggio è critical path più breve . Inoltre disaccoppio il mio circuito in 5 sotto circuiti -> aumento frequenza clock -> diminuisce tempo esecuzione. Andiamo a vedere lo schema base :



Ogni fase è un circuito a se stante , quindi vengono disaccoppiati da dei registri tampone. In dettaglio :



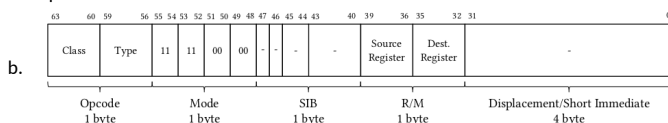
In generale lo schema per ogni istruzione è il seguente :



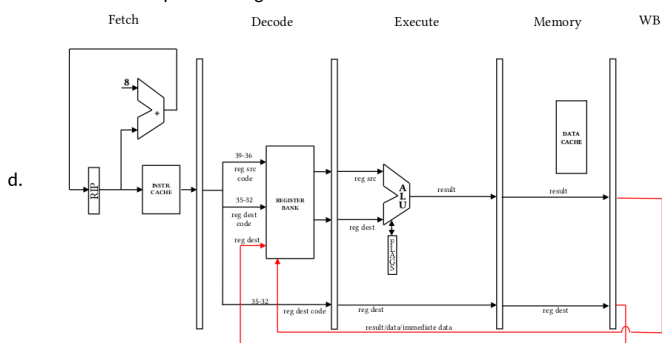
Andiamo a vedere effettivamente come viene implementato il processore per ogni tipo di istruzione :

1. Logico/aritmetico

- a. Vincolo : operandi soltanto a 64 bit , non spiazzamento e dati immediati , solo registri e spiazzamento massimo a 64 bit

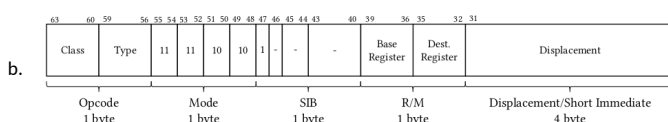


- b. Lo schema completo è il seguente :

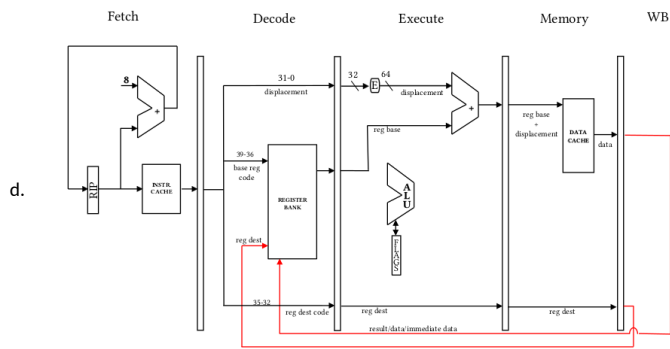


2. Istruzione di load :

- a. Esiste registro base , non uso costanti , non memoria , operandi a 64



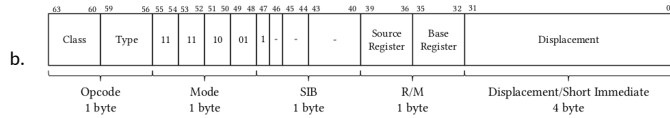
- b. Lo schema completo è il seguente:



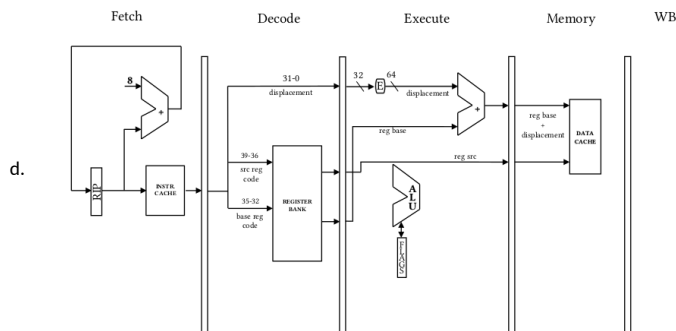
e. Notiamo che la "E" sta per estensione segno : porta da 32 a 64 bit

3. Store :

a. Sorgente registro , dst registro



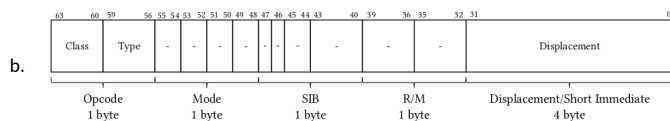
c. Il circuito è il seguente:



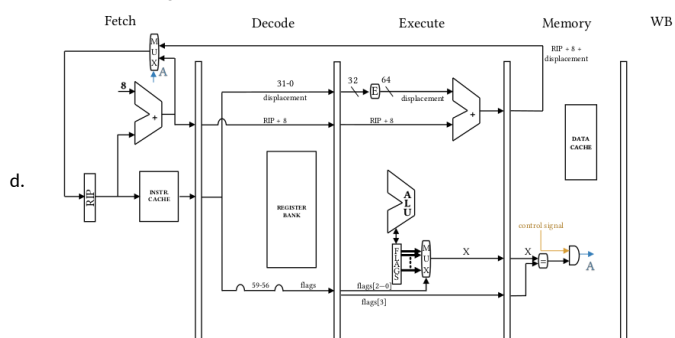
e. Il write back è sprecata : ci passo ma non faccio nulla

4. Salto condizionale :

a. Spiazzamento , bit del registro flag se salto deve essere fatto o meno



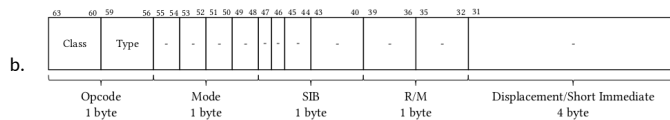
c. Il circuito è il seguente:



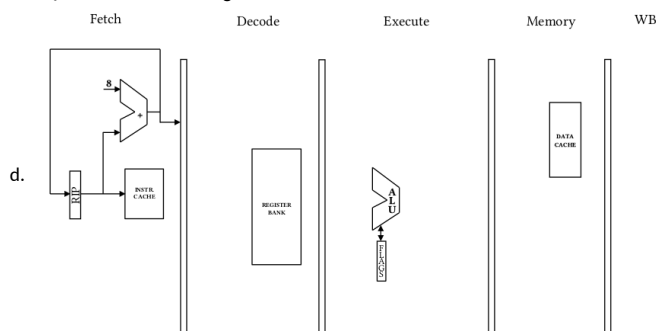
e. Il salto viene scoperto solo nello stato di execute (2 stati dopo) rispetto a Rip->Rip+8

5. Nop

a. 2 fasi

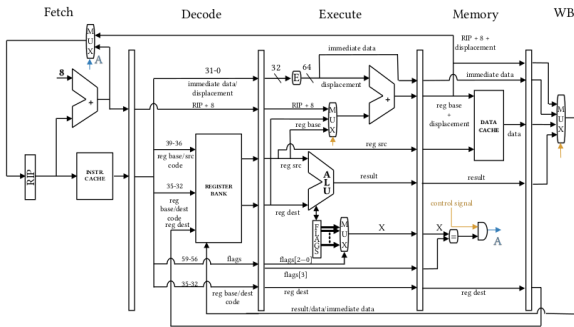


c. Quindi il circuito è il seguente :

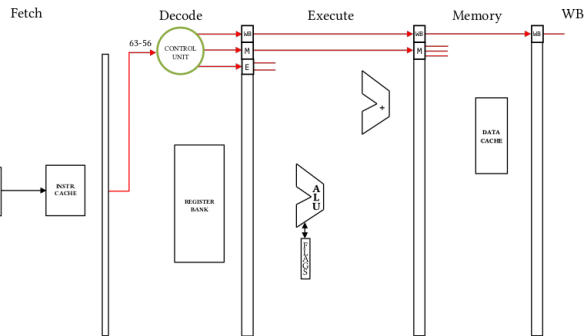


e. Notiamo che sprechiamo le altre fasi !

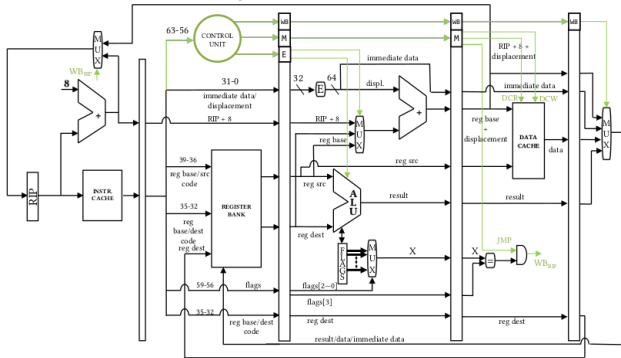
Andiamo ora a vedere il datapath completo : da notare che utilizziamo dei mux per selezionare il valore corretto (da propagare) :



Notiamo che per ogni stadio ci sono 3 segnali di controllo. Quanto detto finora però non tiene conto dell'unità di controllo, la quale serve per vedere come si propagano i segnali tra le varie tubazioni (registri di interfaccia) : vediamo come circuito combinatorio che calcola delle funzioni booleane:



Quindi lo schema finale è il seguente:



Come già detto, la propagazione delle istruzioni con i relativi segnali nei registri tamponi, potrebbe portare a delle problematiche, come per esempio il risultato di un'operazione potrebbe influenzare le operazioni successive, oppure lo stesso risultato potrebbe non essere disponibile quando effettivamente serve. Vediamo un esempio :

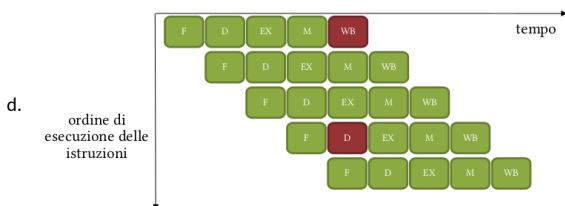
```
addq %rax, %rbx
movq %rbx, 0xabcd(%rdx)
```

In generale questi conflitti (alee) sorgono nei processori pipeline, quando non è possibile eseguire un'istruzione nel ciclo immediatamente successivo senza dovere cambiare la semantica del programma eseguita in una struttura /processore multi ciclo.

Principalmente le criticità vengono suddivise in 3 categorie :

1. Strutturale

- Diverse istruzioni cercano di usare la stessa risorsa hardware, nello stesso ciclo di clock
- Mediato dalla struttura Harvard, la quale, usando la cache di primo livello (vicina al processore), elimina automaticamente la criticità strutturale per accesso a memoria dati e memoria istruzioni.
- Un componente che ne soffre è il banco dei registri, in quanto potrebbe capitare che differenti istruzioni lo usano, ma semantica diversa



- Quindi una possibile soluzione a criticità di questo livello è fare scrittura dei dati sul fronte di salita del clock, mentre la lettura del dato viene effettuata sul fronte di discesa.

2. Su dati

- Criticità nella quale si cerca di usare il dato prima che sia disponibile (magari in seguito al verificarsi o meno di una condizione)
- In dettagli si ha quando un'istruzione nella pipe dipende da una precedente che ancora non ha finito
- Si dividono in due sotto categorie : **define use e load use**
- Vediamo la prima :

- ```
addq %rax, %rbx
subq %rax, %rbx
```

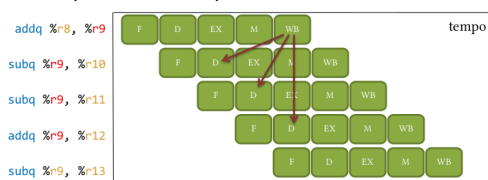
- Qui si nota che lo stesso operando viene usato da due istruzioni : la seconda lo vuole usare quando ancora la prima istruzione non è stata ritirata dalla pipe

- Andiamo a vedere un esempio pratico:

- Possibile coinvolgimento di più istruzioni

```
addq %r8, %r9
subq %r9, %r10
subq %r9, %r11
addq %r9, %r12
subq %r9, %r13
```

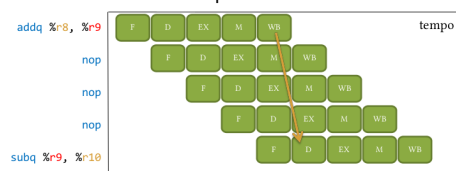
- Che dal punto di vista del processore diventa :



- Questo conflitto può essere risolto in modo sw oppure hw: nel primo inserisco dei tempi morti (aumentando il throughput), oppure sposto le istruzioni (magari vedendo quelle innocue), mentre nella seconda inserisco delle bolle, propagando alla fase successiva i dati appena disponibili

- Vediamo quelle sw:**

- Inserimento istruzione nop

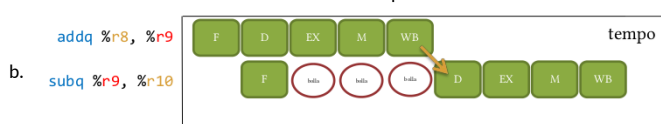


- In generale se criticità strutturale sul banco di registri, bastano 2 nop
- Spostamento delle istruzioni innocue : istruzioni correlate troppo vicine

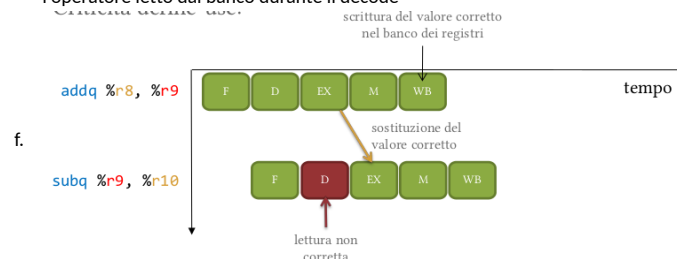


- Vediamo ora quelle hw:**

- Vediamo l'inserimento di bolle : si blocca il flusso nella pipeline, finché la criticità non viene risolta. Vediamo un esempio :



- Quindi le bolle bloccano la fase di fetch, facendo propagare (nella fase di decode) il codice operativo della nop invece che quello dell'istruzione
- Come si individua un conflitto : grazie ad un circuito combinatorio **hazard detection unit**, il quale verifica l'identificativo dei registri di src e dst e verifica che istruzione in uno stato più avanti abbia una fase di write back.
- Vediamo ora la propagazione in avanti : il risultato generato dalla alu viene messo a disposizione dell'istruzione che segue, sostituendo l'operatore letto dal banco durante il decode



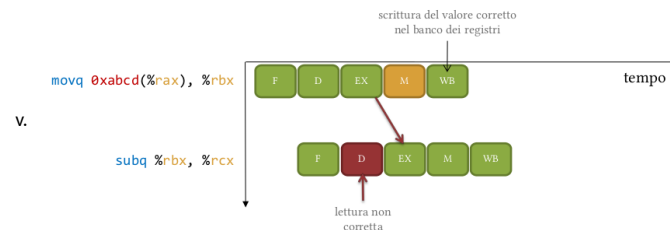
- Vediamo la seconda :

- ```
movq 0xabcd(%rax), %rbx
subq %rbx, %rcx
```

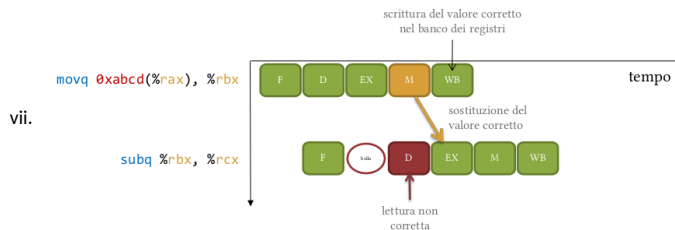
- Qui uno degli operandi della seconda istruzione dipende dal valore letto dalla memoria, la quale ancora non è stata acceduta

- Stesse soluzioni del define use**

- iv. Nel caso di bolle la soluzione è identica , mentre quella per la propagazione in avanti è differente : il dato corretto è disponibile solo al termine della fase di memory:

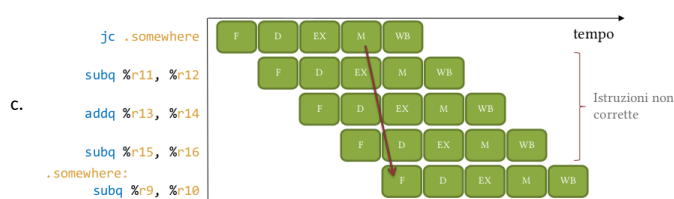


- vi. Da notare che qui il risultato non è corretto : il dato viaggia tra entrambe le fasi di execute : quindi devo inserire una bolla :



3. Su controllo

- Si cerca di prevedere quale sia la prossima istruzione da eseguire, senza avere valutato effettivamente la condizione.
- In questo tipo di criticità , la pipeline viene alimentata effettuando il fetch della prossima istruzione (RIP->RIP+8), e se vi è presente un salto condizionato RIP viene aggiornato durante la fase di memory :



- d. Per risolvere questa criticità si usano 3 tipi di approcci :

- Pessimistico:

- Non si eseguono istruzioni significative fino al completamento della scelta , quindi si ha un'implementazione più semplice , ma vista l'attesa , si ha un aumento del numero di cicli di clock (CPI aumenta)

- Ottimistico :

- Si prospetta che il salto non venga eseguito : quindi se il salto non verrà eseguito aumentano le prestazioni , altrimenti si ha la condizione del pessimistico. Invece come contro si deve annullare (per forza) gli effetti delle istruzioni eseguite dopo il salto : aumenta quindi la complessità dell'unità di calcolo . Vediamo un esempio : il salto viene effettuato: quindi si deve annullare (nella pipeline) l'esecuzione delle istruzioni entrate nella pipe : **flash della pipe**, quindi ripristino dei registri , simulando così una nop , **ed in particolare si deve usare altro registro tampone per salvare la copia del registro flags**.

- e. Quindi in ottica di ottimizzare le prestazioni (sempre caso del salto) : se salto si cerca di prevedere se il salto sarà effettuato o meno : se effettuato si massimizzano le prestazioni , mentre nel secondo si effettua il flush della pipe . In generale due tipi di predizioni sulla condizione : **dinamica e statica**. Vediamoli in dettaglio : il primo si ha quando il valore previsto cambia durante l'esecuzione , ma necessita di implementazione hardware (branch prediction unit) , e questo salto sarà effettuato basandosi sullo storico ; mentre per la seconda il salto verrà effettuato verso e solo parti precedenti del codice , e questo tipo di predizione viene guidata dal compilatore e dal programmatore. Quindi riassumendo: per la predizione del salto , in generale si usa la cosiddetta **tabella di predizione del salto** : ovvero piccola memoria indicizzata dal bit meno significativo dell'indirizzo dell'istruzione di salto condizionato , nella quale ad ogni istruzione contenuta nella memoria è associata una previsione: effettuo il salto oppure no?? : nel primo caso il salto si effettua e si paga un solo ciclo di clock in più , mentre se nella seconda non si fa , quindi non si sprecano cicli , mentre nel caso peggiore (previsione errata), la si deve modificare , si deve effettuare il flush della pipe e per quanto riguarda la penalità è la stessa se non vi fosse predizione. In generale per prevedere se il salto si farà , si potrebbe usare una macchina a stati finiti : la quale effettua il cambio di previsione dopo due previsioni errate :

